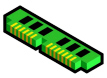
For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

How to design a backend cache

Do we always want cash?

Yes.



Do we always want to cache?

It is not always so.

Pros

1. Caching improves read throughput (assuming high hit rate)

Throughput: *the average number of tasks completed per unit time. High throughput is good.*

Cons

1. add complexity
    - has to be designed
    - additional infrastructure to maintain
    - in the app server code, the caching logic gets mixed in the business logic
        - one way to mitigate this is to move the caching to the middleware
2. Increases latency (if there's a cache miss, you now have to check 2 places, first cache, then DB)

(True) ***Latency:*** *the amount of time a request spends waiting to be processed*
(Colloquial) ***Latency:*** *the amount of time it takes for the response to come back*

5 step process to cache design

1. Establish the need for caching
2. Determine the type of cache
    a. Local vs Global
    b. Single vs Distributed *(only if global cache)*
3. Identify the Invalidation algorithm based on

    a. consistency requirements
    b. data/query complexity
4. Identify the Eviction algorithm
    a. just use LRU
5. Think about the load balancer

Caching Case Study - 1

Scaler Code Judge

Code Judge — service that "judges" your code submission. When you submit a solution for a DSA problem in Scaler, we have to take your code and run it through hundreds of testcases to evaluate whether it is correct or not.

Situation

Q: What does the appserver need to evaluate a code submission?

1. User info (auth, already solved this problem or not, part of course or not, ..)
2. Problem info (topic, total score, memory limit, time limit, number of testcases, ..)
3. ***Testcase data*** *(input file, expected output file)*

Q: How large can this data be (in the avg/worst case)?

User info / Problem info – `a few KBs` ***(SQL database - Amazon RDS)***
Testcase data – `1 to 2 GBs` ***(File storage - Amazon S3)***

For a problem we run your code through `100` testcases.
Imagine that the problem is about sorting the array.
Each testcase comprises of a large array $(N = 10^6)$
Each expected output for each testcases is also a large array $(N = 10^6)$

$$100 \text{ testcases} * (10^6 \text{ integers / testcase}) * (8 \text{ bytes / integer})$$
$$= 100 * 10^6 * 8 \text{ bytes}$$
$$= 800 \text{ MB}$$
$$\sim 1 \text{ GB}$$

This is for the input file.. similar size for the output file.

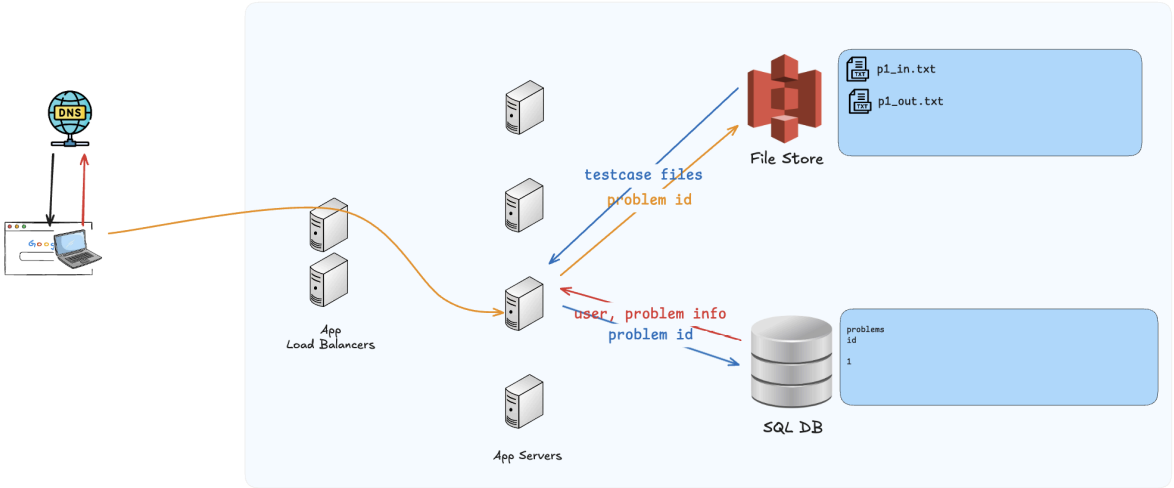
Q: What is the total size of testcase data in S3?

Scaler has around `3,000` problems
so around `3000 problem * (1GB testcase data / problem) = 3TB` of testcase data

Is this data too large?
Not really - storage is cheap.
3TB is too large to fit completely in RAM. But can easily fit on HDD.

Q: How many problems are being solved on any given day?

Around `100`
We're running multiple batches in parallel. Each batch might be solving the problems of the current (or last) class. The batches also overlap in timelines.



Q: Should we store these testcases in a CDN?

No! Backend services will *never* access data from the CDN

1. CDNs are client facing
2. CDNs reduce latency for the client, by serving the data from an (edge) server close to the user

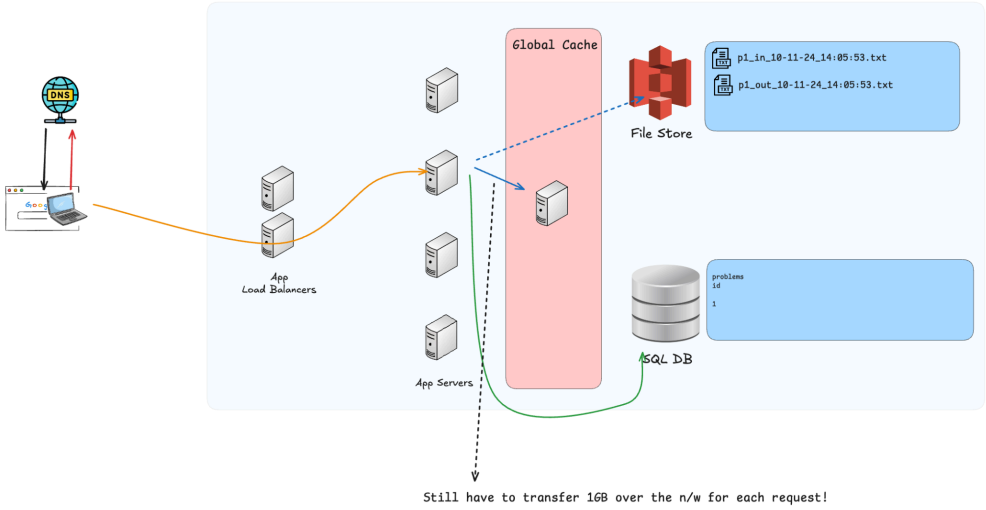
Need for caching

Q: Do we want to transfer 1GB of data over the network for each request?

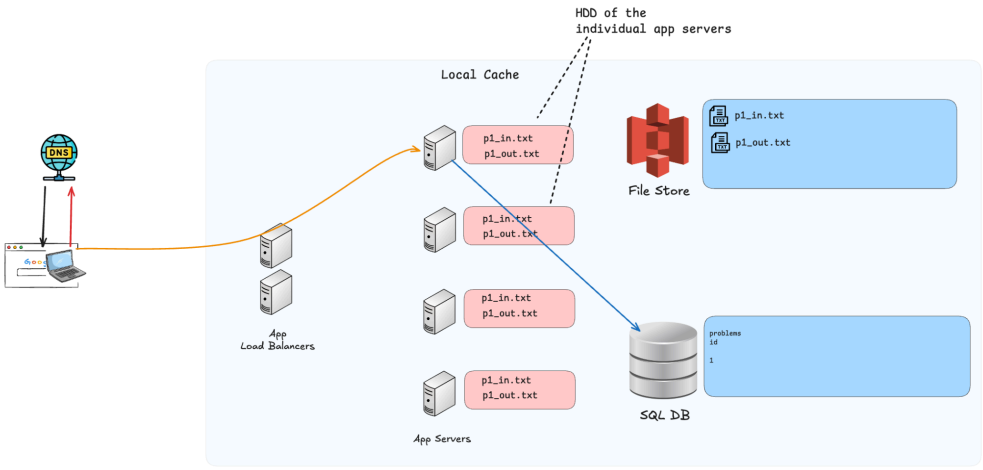
Absolutely No.
Therefore, we need a cache!

Local vs Global

- **Global Cache:** doesn't really solve our problem
  - yes, it will be available to all app servers (so our app servers can be stateless)
  - yes, the data is large, so global makes more sense?
  - NO - it doesn't solve our problem at all - we still have to transfer 1GB of testcase data for each request - from the cache server to the app server
    - earlier we were transferring from S3 to app server – expensive!
    - now we're transferring from cache to app server – expensive!



- **Local Cache:** each server caches some testcases in the local HDD
  - the request will come the app server
  - the app server will hit the SQL db to fetch the user/problem info (small data)
  - if the problem's testcases are already cached in the appserver's HDD, it will just use those testcase files (no n/w overhead)
  - if not cached, it will download the files from S3, and cache it locally (on its HDD), and then use it
    - any subsequent requests for the same problem will now use the testcases from the HDD



POST /evaluate-submission

```
fn check_solution(solution):
    pid = solution['problem_id']
    user_info, problem_info = fetch_from_SQL(user_id, pid)

    if not file_exists(`{pid}_in.txt`):  // caching logic
        download_from_s3([`{pid}_in.txt`, `{pid}_out.txt`])
        // this function will save it on the local HDD

    inputs = read_file(`{pid}_in.txt`)  // these reads are from the local HDD
    expected_outputs = read_file(`{pid}_out.txt`)

    return evaluate(solution.code, inputs, expected_outputs)
```

Single vs Distributed

N/A - single vs distributed only applies for global caches
Local cache is distributed automatically, because there are multiple app server

Invalidation Algorithm

When is invalidation applicable? if the data never changes, do we need to invalidate?
Invalidation is only applicable when the data changes.

Q: Can the testcases change?

~~New problems can be added - but that is not a data updation.~~

Can the testcase data for an existing problem be updated?
Yes.

Because our problem setters are not Gods.. they can make mistakes.
It is possible that

- our editorial solution was incorrect
    - we've to fix the solution and the `expected_output.txt` file
- our solution was correct, but our testcases were not "tight enough"
    - our problem required a `O(n log n)` solution, but the testcases let the $O(n^2)$ solutions pass as well.
    - so we've to fix both the `input.txt` and the `expected_output.txt` file

Q: How frequently will the testcases change?

Extremely rarely. We have good problem setters, and a lot of quality assurance to ensure that bad problems are not created.
But it happens from time to time.

Let's say for a given problem we might want to update the testcases once a year on average.
`1 update / problem / year on average`

Q: Do we really need to worry about this edgecase?

Can we just ignore this, since it is a rare edgecase?
Absolutely NOT!

We must have the infrastructure & logic in place!

Imagine that we realize that our testcases are bad during a live contest — we can't say stuff like "oh but I thought that this will be so rare, my cat also thinks the same, so I didn't implement this…"

So we need to have an invalidation algorithm in place.

We know that different invalidation techniques give us different consistency guarantees.

Q: What consistency do we require?

Q: Is eventual consistency good enough, or do we require immediate consistency?
Q: What does eventual consistency mean in this context? What does it mean to have a stale read?

Imagine that during a contest, we realise that some testcases are wrong. Our problem setters will create new testcases, and upload them to S3. And we announce to all contest participants that the testcases have been updated, please resubmit problem 3.

Eventual Consistency: after updation & announcement, for sometime (next 10 mins) when people resubmit P3, still, the old testcases are used. **NOT good enough!**
Immediate Consistency: after updation & announcement, any submissions to P3 use the new testcases (guaranteed!)

So now that we know that we require immediate consistency, it is obvious that we should use Write Through.

But, let's explore - just for learning

~~TTL~~

Every time we download the file from S3, we maintain an expiry time. If the request comes before expiry, we will use these cached testcases, otherwise, we will assume that the file has expired, we will delete the cached testcase, and fetch again from S3

```
fn check_solution(solution):
    pid = solution['problem_id']
    user_info, problem_info = fetch_from_rds(pid)

    if file_exists(`{pid}_in.txt`):
         if read_last_updated_at(`{pid}_in.txt`) < now() - (1 hour):
              // file was downloaded more than 1 hour ago — stale!
              delete_file(`{pid}_in.txt`)    // TTL invalidation

    if not file_exists(`{pid}_in.txt`):         // caching logic
        download_from_s3([`{pid}_in.txt`, `{pid}_out.txt`])
        // this function will save it on the local HDD

    inputs = read_file(`{pid}_in.txt`)    // these reads are from the local HDD
    expected_outputs = read_file(`{pid}_out.txt`)

    return evaluate(solution.code, inputs, expected_outputs)
```

What is the ideal TTL?

- 1 week?
- 1 day?
- 10 hours?
    - *by the time the TTL expires, the contest is over. The updated testcases never took effect*
- 1 hour?
- 10 min?
    - *for the next few minutes after your announcement, the old testcases are still being used. Bad user experience*
- 1 min?
- 10 seconds?
- 1 second?
    - *every 1 min the TTL is expiring. After 1 min, each app server will have to re-download 5-10 GB files from S3.*
        - *if we've 1000 app servers for code judge, then we will be transferring 5TB of data per minute*
    - *defeats the purpose of caching*
        - *the miss rate is too high*

There should some sweet spot for TTL.

No! There's absolutely no sweet spot - none of the values are going to work. All of them are bad experience some way or another.

~~Write Around~~

Same as TTL — we will have to decide how frequently the CRON job will run.
Once again, there's no ideal value

~~Write Back~~

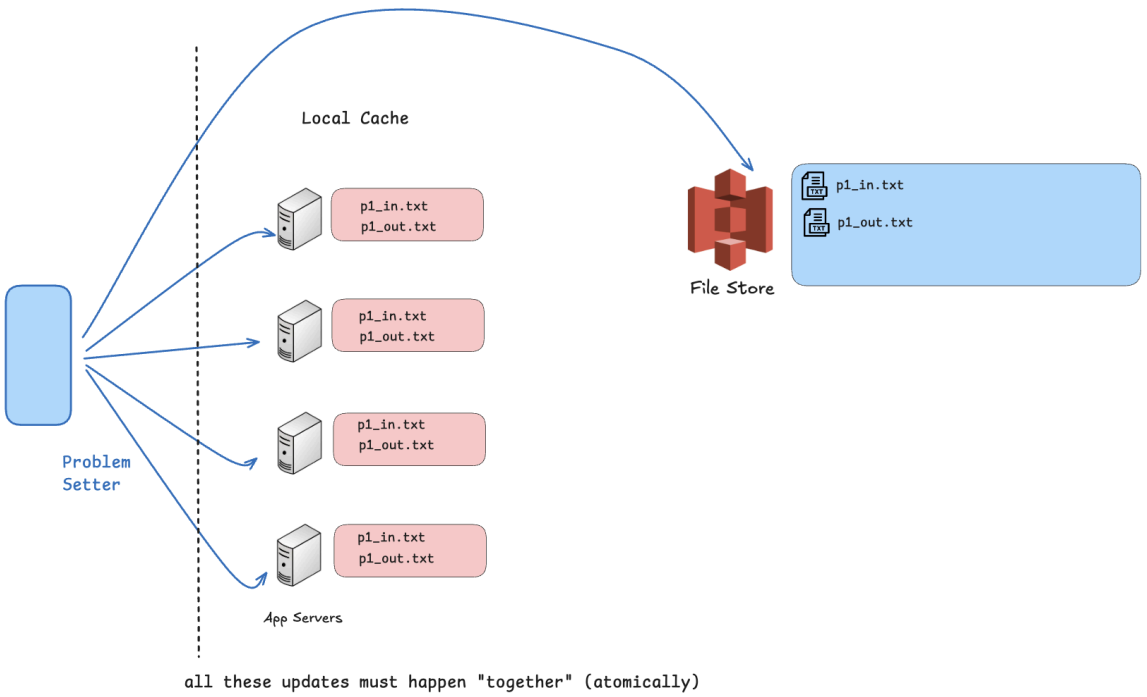
This is stupid — this will lead to data loss. Additionally, we've a local cache - we will have to write to 1000 app servers whenever a write comes.

Write Through Cache

Q: it is feasible to update 1GB files in hundreds of app servers + S3 in an atomic manner?

Absolutely not!
Maintaining atomicity across 2 servers is already insanely hard & slow.
Maintaining atomicity across 100s of servers is impossible

- at least 1 server will fail - you now have to rollback the other 99 servers
- you will be limited by the slowest server (incredibly slow writes)



this will NOT work!

Instead, we can do the following

1. we will version the testcase files using a timestamp in the file name
    - `p1_input.txt` ⇒ `p1_input_2025-03-16 08:00.txt`
2. the normal cache works as earlier *(with slight modification)*
    - when a request comes to the app server, we first check the SQL db to fetch the user info and problem info
    - **during this call itself, the SQL db will also give us the input/output file names for this problem's testcases**
    - if these files are already available in the HDD (local cache), then just use them
    - if not, then fetch from S3 and then use them
3. **for invalidation**
    - problem setter will upload the new files ***(with new timestamp in name)*** to S3
    - problem setter will update the file name in SQL db
        - note that these two writes (S3, SQL) don't have to be atomic.
        - we can first upload to S3, and only if that succeeds, we can update the filename in SQL
        - because if the new file is uploaded in S3, but filename is not updated in SQL, the new is not causing any harm – it's just not being used.
    - for any requests that come to the app server after the SQL entry has been updated
        - the SQL db will provide the new file name
        - app server will detect that it does not have these new files in the HDD (Local cache)
        - so, the app server will fetch these files from S3 & then use them

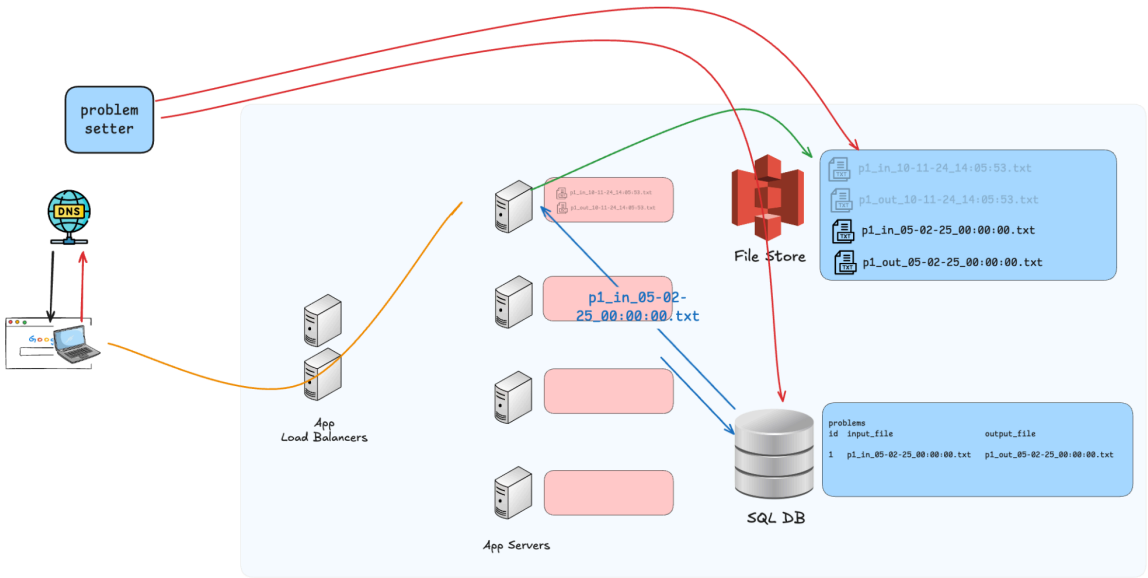
```
fn check_solution(solution):
    pid = solution['problem_id']
    user_info, problem_info = fetch_from_rds(pid)

    file_name = problem_info['file_name']    // get the version from the SQL DB

    if not file_exists(`{file_name}_in.txt`):                          // caching logic
        download_from_s3([`{file_name}_in.txt`, `{file_name}_out.txt`])
        // this function will save it on the local HDD

    inputs = read_file(`{file_name}_in.txt`)                  // these reads are from the
local HDD
    expected_outputs = read_file(`{file_name}_out.txt`)

    return evaluate(solution.code, inputs, expected_outputs)
```



Basically, we're caching the testcases
we're not caching the version id - the version id is fetched everytime from the DB — so version id cannot be stale

Now that we're not updating the testcases at all (we're not modifying existing files, we're uploading a new version), the testcase are immutable - since they never change, there's not need for invalidation.

Eviction Algorithm

LRU eviction

Operating system will automatically maintain the read timestamps for all files. We can just use that for LRU eviction.

We can find all the files in the folder — whichever file was least recently used, just delete that to make space.

```
fn check_solution(solution):
    pid = solution['problem_id']
    user_info, problem_info = fetch_from_rds(pid)

    file_name = problem_info['file_name']    // get the version from the SQL DB

    if not file_exists(`{file_name}_in.txt`):      // caching logic
        if get_folder_size('.') > 100GB:
            delete(get_oldest_accessed_at_file('.'))    // LRU eviction
        download_from_s3([`{file_name}_in.txt`, `{file_name}_out.txt`])
        // this function will save it on the local HDD

    inputs = read_file(`{file_name}_in.txt`)                  // these reads are from the
local HDD
    expected_outputs = read_file(`{file_name}_out.txt`)

    return evaluate(solution.code, inputs, expected_outputs)
```

Load Balancer

The LB can just use **Round Robin**. Each app servers acts independently.

Should we use consistent hashing?

- Round Robin
  - any request can go to any app server
  - All app servers will have to cache all 100 problems that are being solved on any given day.
  - This seems like a waste (but is not).
- ~~Consistent Hashing based on user id~~
  - same thing
  - because any user can be solving any problem, and multiple users will be assigned to any server,
  - every server will have to cache all problems that are being solved today
- **Consistent Hashing (based on problem id)**
  - any requests for Problem 1 goes to server 1
  - any requests for Problem 2 goes to server 2,
  - ...
  - Now only server 1 has to cache the testcases for problem 1.

But, routing based on `problem_id` is a bad design!

1. during a contest, 100k people are solving 5 problem
2. out of the 100 app servers, only 5 are getting all the load - **uneven load distribution**
3. in our case, a single server can't even handle multiple requests at the same time

a. typically, to handle multiple requests at the same time - we use multi-threading

Q: Is multi-threading a good idea in this case?

NO! We can not multi-thread the code-judge!

Code evaluation is a CPU bound task

Q: What does multi-threading do?

Most programs are I/O bound. They're waiting for some I/O to happen (user keystroke / mouse click, network download, file read, printer access / ...)
CPU are millions of times faster than disks / networks.
99.99% of the time, your CPU is idle.
So, you can do multiple things at the same time by "context switching" the CPU.
Do task 1 - now that task 1 is waiting for some I/O - but instead of waiting, you context switch start doing task 2 - task 2 will also go for I/O - context switch back to task 1
do task 1 - ...
...

CPU bound tasks (video processing, heavy computation, analytics, machine learning, sha calculation, ...)
context switching will worsen the performance - because the CPU is already busy - if you try to break its loop and get it to multi-task it will slow everything down - thrashing

Moral: only multi-thread I/O bound processes. Never multi-thread CPU bound processes.

In code judge, evaluating a single request requires ~5 seconds. During those 5 seconds the app server is completely occupied - it cannot handle any other requests.
Therefore, we want requests to go to the next available server - Round Robin routing

Caching Case Study - 2

Scaler Contest Leaderboard

Leaderboard

- paginated

- ranklist
- for each rank, it shows
    - user details
    - for each contest problem
        - problem details
        - user's score & submission details for that problem

Type to search & press enter | 25 | Institution | e.g. Indian Institute of Technolo | Apply

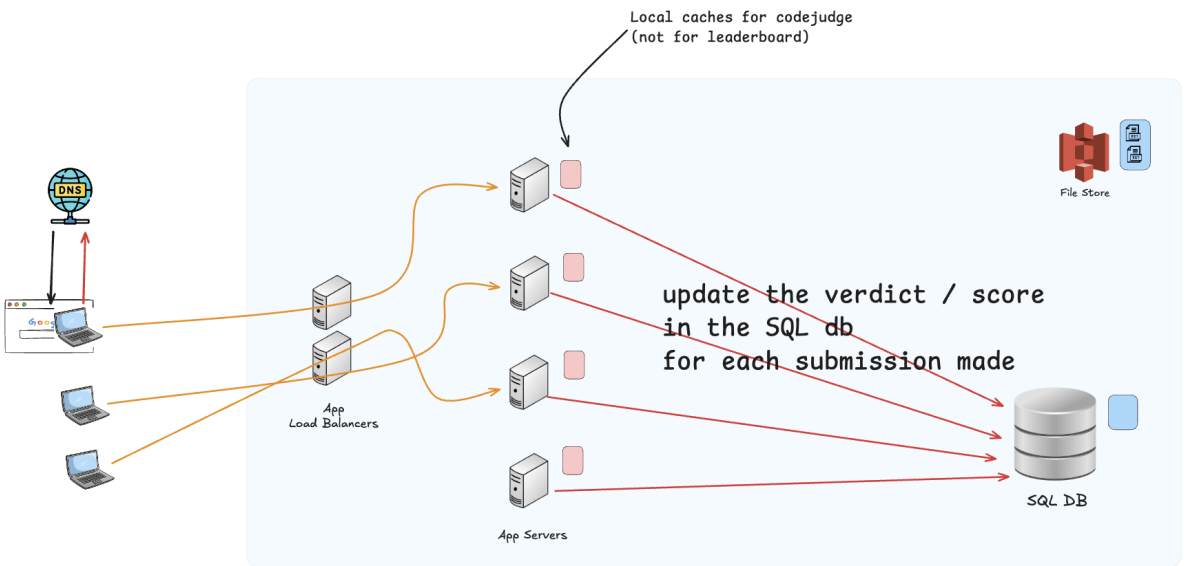
#	USER NAME	SCORE	SPCTRIPS	ARRFILL	GEO1	CHFINVNT	ALTLG	CHARGE	CDIGL
1	7★ sam_rei Not Available	1000	100	100	100	100	100	100	100
1	7★ Ra16bit Other	1000	100	100	100	100	100	100	100
1	7★ anshugarg12 Indian Institute of Technology B...	1000	100	100	100	100	100	100	100
1	6★ rns6 Kim Il-sung University	1000	100	100	100	100	100	100	100
1	6★ nishant403 Dhirubhai Ambani Institute of In...	1000	100	100	100	100	100	100	100
1	7★ rgb_icpc2 Not Available	1000	100	100	100	100	100	100	100
1	7★ oilhun Pyongyang University of Scien...	1000	100	100	100	100	100	100	100
1	6★ scheffeli Not Available	1000	100	100	100	100	100	100	100
1	7★ xams Montgomery Blair High School	1000	100	100	100	100	100	100	100
1	6★ dqhdl Trường THPT chuyên Thăng L...	1000	100	100	100	100	100	100	100

Situation

- 3 hour contest
- 100,000 participants
- 5 problems

Users are submitting their solutions at very high rate (100s of submissions per second)

These submissions are being evaluated by the code judge (as discussed in the previous case study)
For each request, the code judge will update the final verdict/score in the SQL db.
Based on this collective data, we need to compute and show the leaderboard.



Q: How can we compute the ranklist?

What contest is running (`contests` table)
What users are participating in this contest (join b/w `users` table and the `contest_participants` table)
What problems are there in this contest (join b/w `problems` table and the `contest_problems` table)
What submissions have the users made for this contest (join b/w `users` table, `user_submissions` table, `problems` table, `contest_problems` table)

We will take all this data
- aggregate this by user_id
- calculate the final score for each user (based on the individual submission scores, the number of incorrect attempts, the amount of time they took to solve it, ...)
- sort the users by the final score

Q: How much data is there?

Imagine that each user makes `5 submissions` (on average) over `3 hours` of the contest.
- 90% users will give up in just problem 1
- 10% users might make multiple attempts at each problem

```
total number of submissions = 100,000 users * 5 submissions / user
= 500,000 submissions
```

if each submission detail is `100 bytes (problem id, score, time taken, verdict, user id, contest id, ...)`

```
total size = (100 bytes / submission) * 500,000 submissions
= 50 MB
```

Q: How much time does it take to compute the ranklist?

because this is heavy compute, lets say it takes us ***~5 seconds*** for us to compute this ranklist

Need for caching

Q: How frequently is the ranklist queried?

Assume that each user views the ranklist `20 times` (on average) during the contest

```
Requests / second
= (20 views / user / 3 hours)  * 100,000 users
= (20 views / 3 hours) * 100,000
= 2 million views / 3 hours
= 2 million views / (10,000 seconds)
= 200 views / second
```

Q: Can we calculate the ranklist 200 times / second, when it takes 5 seconds to compute the ranklist once?

That's stupid.

Because the request rate is very high, and the ranklist computation is heavy, we don't want to compute it again and again.
Hence the need for caching.

Local vs Global

Q: What does a typical query look like?

Either of

- show me page 20 (ranks 200 to 210)
- show me my rank

Q: What does the data (query result) look like?

A simple JSON file

```
[
    {rank: 1, user_id: ..., user_name: ..., problems: [{1: ...},
{...}, ...] },
    {rank: 2, user_id: ..., user_name: ..., problems: [{1: ...},
{...}, ...] },
    {rank: 3, user_id: ..., user_name: ..., problems: [{1: ...},
{...}, ...] },
    {rank: 4, user_id: ..., user_name: ..., problems: [{1: ...},
{...}, ...] }
]
```

Q: How large is this JSON response?

```
(100 bytes / user entry) * (10 entries / page)
= 100 bytes * 10 / page
= 1KB / page
```

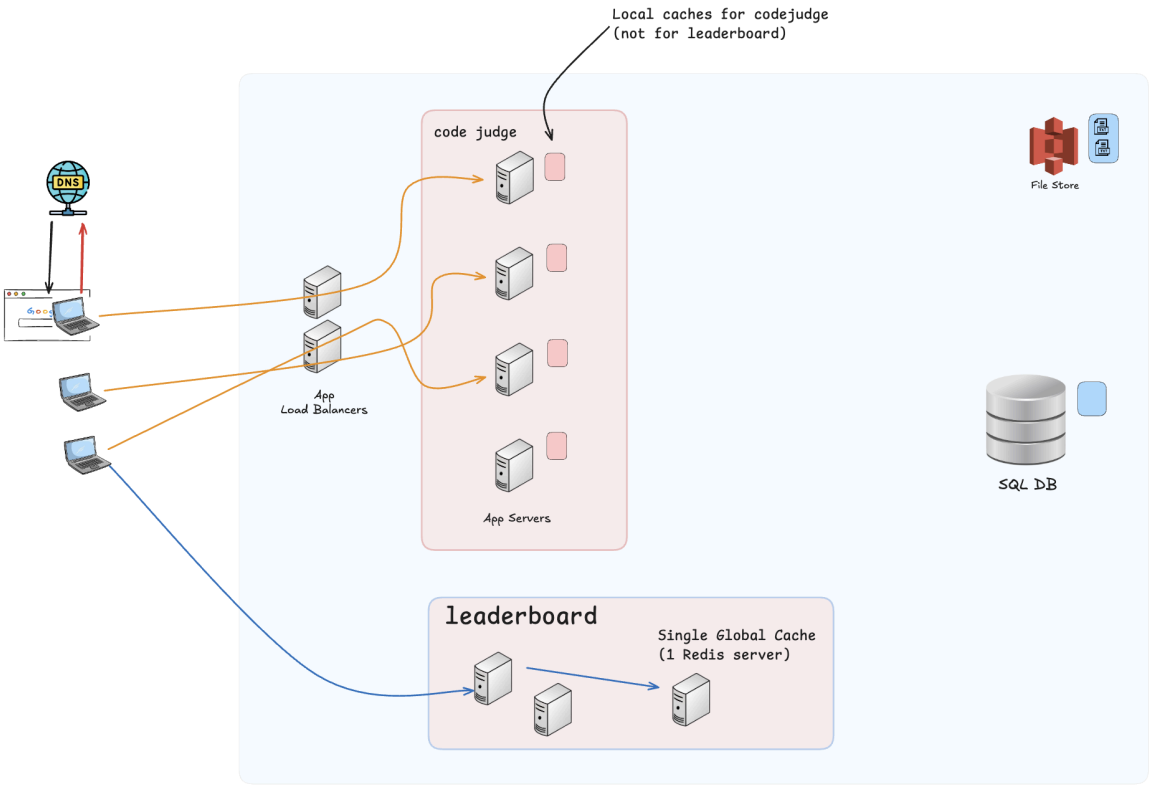
Single vs Distributed

Q: How much total data do we have (in the entire ranklist)?

```
(100 bytes / user entry) * (100,000 users)
= 10 MB
```

this is a tiny amount of data!

Data is tiny, so a single cache server can store it
200 queries / second can easily be handled by a single redis server



Redis

Very fast, in-memory, key-value database

Fast because
- written in C
- single threaded (uses async-io instead of threads)
- lots of low level optimizations

In-memory
- uses the RAM
- (optional - not enabled by default) support for disk-persistence (note that this will reduce write speed)

Key-value
- no complex joins
- no indexes
- no complex queries
- think of it as a hashmap in memory

Database
- but mostly used as a global cache

Mandatory reading

1. **Try online:** https://onecompiler.com/redis
2. **Quick start:** https://redis.io/learn/howtos/quick-start
3. **Cheatsheet:** https://redis.io/learn/howtos/quick-start/cheat-sheet

Optional reading

1. **Tutorial:** https://redis.io/university/
2. **Docs:** https://redis.io/docs/latest/
3. **Eviction policies & Cluster mode:** https://docs.google.com/document/d/1k4nzubvtX_yLctUT4VWK8ZJt4KCcOEdRJdxQgWCaiU8/

Invalidation Algorithm

Q: Is immediate consistency good to have?

It's always good to have. If we can get immediate consistency without any issues, then why not!

Q: Is immediate consistency critical?

No. If the ranklist has changed, but the changes don't show up in the leaderboard for some time - that's not the end of the world.
That won't cause a bad user experience.

Eventual consistency is good enough.

Q: Is immediate consistency possible?

How frequently does the "true" ranklist change?

With every submission!
Suppose users make 5 submissions on average during the contest
Submissions/sec = `5 submissions / user * 100,000 users / 3 hours`
`= 500,000 submissions / 3 hours`
`= 500,000 submissions / 10,000 seconds`
`= 50 submissions / sec`

True ranklist changes 50 times / second
It takes ~5 seconds to compute the ranklist

It is impossible to get immediate consistency.

Eventual Consistency is good enough!

Q: How frequently should the ranklist be invalidated?

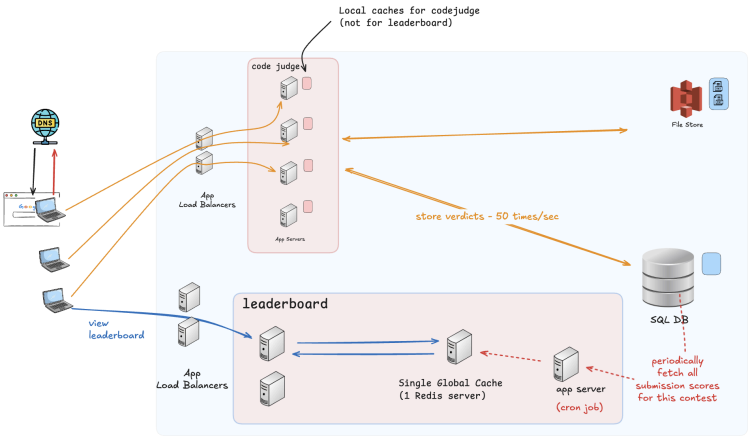
Invalidate it every

- 10 hours
- 1 hour
- ***10 mins***
- 1 min
- 10 seconds
- 1 second
- 100 ms
- 10 ms

In fact, Scaler invalidates the ranklist every 30 mins

Both TTL & Write Around provide eventual consistency.

- ~~TTL~~
    - easy to implement & use
    - we're already using Redis, and redis has support for TTL
    - but the computation is heavy
        - if data is not in the cache, the app server cannot make the user wait for 5 seconds to compute the rank list
- **Write Around**
    - Separate app server which run periodically (every 10 mins)
    - It will fetch all submission scores for this contest
    - Compute the ranklist
    - Store the ranklist in the Redis server



-

What exactly does Redis Store? Ranklist Schema?

Key	Value
contest:3:page:1	[{rank: 1, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, {rank: 2, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, {rank: 3, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, {rank: 4, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, ... 10 entries]
contest:3:page:2	[{rank: 11, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, {rank: 12, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, {rank: 13, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, {rank: 14, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }, ... 10 entries]
...	
contest:3:page:10000	[{ ... }, { ... }, { ... }, { ... }, ... 10 entries]
contest:3:user:377	{rank: 1, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }
contest:3:user:377	{rank: 1, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }

`contest:3:user:377`	`{rank: 1, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }`
`contest:3:user:377`	`{rank: 1, user_id: ..., user_name: ..., problems: [{1: ...}, {...}, ...] }`
... 100,000 entries (one for each participant)	

Total amount of data in Redis = `50MB + 50MB = 100MB`
(because each user's entry is being stored twice *(once for page, once for the user)*)

Eviction Algorithm

100MB of data / contest. There's no need of eviction!

Note that we will have to store this data for every live contest.
Once the contest is over, the ranklist for that contest can't change (no more new submissions) - so the ranklist can just be dumped in the SQL db itself (no need to cache data that doesn't change)

How many live contest might we run in parallel on any given moment?
Max 10 (usually 1 or 2)

1GB of data in total => still doesn't require eviction

Load Balancer (for cache)

There's no LB for the cache - the cache is a single Redis server

If the cache server crashes - no data loss (cache doesn't store any "real" data)
We will just restart the cache server, and the cron job will run automatically after 10 mins (or we can force it to run after the cache server has been restarted)

Next Class

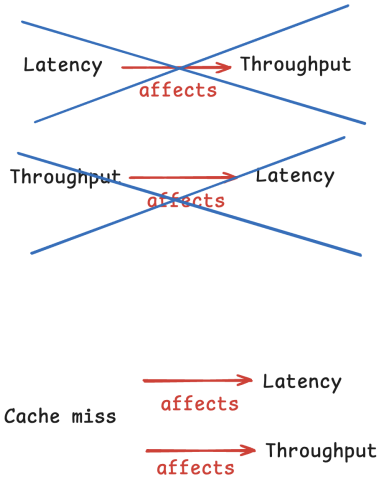
Caching Case Study - 3 - Facebook News Feed

Doubts

Latency vs Throughput

These are completely “orthogonal” concepts – latency is causally independent of throughput. High throughput doesn’t necessarily mean low latency. High latency doesn’t necessarily mean low throughput.

Typically these are correlated because of some other causal link.



ChatGPT — when you ask it to generate a Ghibli image, it takes a long time to generate. Latency is high. But is the throughput low? No.. because it is generating millions of images in parallel! The number of images generated per second is still amazingly high – it’s just that each image takes a long time to generate (because of diffusion)

Communication is the backbone of society

If you’re not to communicate effectively, it doesn’t matter how intelligent you are, it doesn’t matter how educated you are, it doesn’t matter how hard working you are.

Communication is key!

Communication needs to be practiced correctly.

Making broken sentences shows poor communication – and this will cause an immediate reject in interviews!